

Submit any 5 questions & practice on others later.

Part 1: Nested Queries & Aggregations

1. Write a query to display the names of all Patients who have been infected with the 'Dartu Virus'. Use a subquery to find the GermID.
 2. List all Germs that have an ImpactLevel higher than twice the average ImpactLevel of all germs in the database.
 3. Use an EXISTS clause to find all Locations that have at least one germ currently associated with them in the Germ_Locations table.
 4. Display the names and types of all Weapons that target germs discovered before '2024-01-01'.
 5. Write a query to list each Location name along with a count of how many germs are associated with it. Perform the count using a correlated subquery in the SELECT list.
 6. Find the names of Patients who were prescribed 'Super Soap'. You must nest queries through Prescription_Remedies and Remedies.
 7. Find the names of Patients who have been infected by both 'Airborne' and 'Waterborne' germs using a nested intersection logic.
-

Part 2: Views

8. Create a view named vw_HighImpactGerms that displays all details of germs with an ImpactLevel of 8 or higher.
 9. Create a view named vw_PatientPrescriptions that shows Patient Name, Prescription Date, and the Remedy Name they were prescribed.
 10. Create a view named vw_LocationRisk that lists each Location name and the total number of infections reported there (based on germs found at those locations).
 11. Create a view named vw_PatientPrivacy that shows only the PatientID, Age, and Gender, masking the Name column for data privacy.
 12. Create a view named vw_ActiveOutbreaks that shows Location Name and Germ Name for all infections where RecoveryDate is NULL.
-

Part 3: Stored Procedures

13. Create a stored procedure sp_AddPatient that accepts Name, Age, and Gender as parameters and inserts a new record into the Patients table.
14. Create a stored procedure sp_UpdateImpact that takes a GermID and a NewImpact value. It should update the germ's impact level only if the new value is between 1 and 10.

15. Create a stored procedure `sp_AssignRemedy`. It should accept `PrescriptionID` and `RemedyID`. If the remedy's `EffectivenessLevel` is less than 5, print a warning; otherwise, insert it into `Prescription_Remedies`.
 16. Create a stored procedure `sp_GetPatientHistory` that takes a `PatientID` and prints/selects all germs that patient has ever been infected with.
 17. Create a stored procedure `sp_CheckLocationSafety`. If a location has 0 active infections associated with its germs, update its `InfectionStatus` to 0 (Safe).
-

Part 4: Triggers

18. Create a trigger named `tr_AuditPrescriptions` that automatically inserts a record into `Prescriptions_History` whenever a new prescription is added to the `Prescriptions` table.
 19. Create a trigger on the `Infections` table that prevents a record from being inserted if the `InfectionDate` is in the future.
 20. Create a trigger on the `Infections` table so that whenever a new infection is added, the `InfectionStatus` of all `Locations` where that germ is found is automatically set to 1 (Infected).
 21. Create a trigger that prevents a `Germ` from being deleted if it is currently associated with any `Weapons` in the `Weapon_Germs` table.
 22. Create an `INSTEAD OF DELETE` trigger on the `Remedies` table that, instead of deleting the remedy, sets its `EffectivenessLevel` to 0 to "retire" it.
-

Part 5: Transactions

23. Write a script using `BEGIN TRANSACTION` to insert a new `Prescription` and two `Prescription_Remedies`. Use `ROLLBACK` if any error occurs and `COMMIT` if both succeed.
 24. Write a transaction that inserts a new `Infection` and simultaneously updates the `InfectionStatus` of the corresponding `Location`. Ensure that if the update fails, the infection insert is also reversed.
 25. Write a script using `SET TRANSACTION ISOLATION LEVEL` to prevent "Dirty Reads" while running a report on active infections while updates are occurring.
-
26. Find remedies whose effectiveness level is greater than the impact level of all those germs who have infected patients that have been treated by that remedy in the last 6 months.
 27. Find all those germs who have infected males at least two times more than females in every month and whose discovery date is at least a year before prescription date for at least 50 patients.

28. Find the patients and the count of germ roles such that germs of that role, say 'airborne', have infected patients by age group in the same proportion that they are found in locations. For instance if airborne germs have infected 30% patients in the age group 30-40 and they are found in 30% patients then each patient in the 30-40 age group must be named.
